

MLFF-DATA1 Shared Correlated-Sampling Primitives

mdstats

2026-07-27

Purpose

MLFF-DATA1 introduces the source-independent `mdstats.sampling` foundation used by later training-data partitioning and by existing Stage 11 statistical workflows. It owns four narrowly bounded operations:

1. deterministic integrated-autocorrelation estimation;
2. contiguous complete-frame interval construction;
3. deterministic balanced assignment;
4. neighboring purge and purged-fold construction.

The module does not know about VASP, chemical species, atoms, MACE, physical feature selection, dataset roles, or active learning. Later layers adapt these primitives into source-bound records.

Runtime ownership

```
mdstats/sampling/  
  autocorrelation.py  
    AutocorrelationPolicy  
    AutocorrelationEstimate  
    estimate_autocorrelation  
    integrated_autocorrelation_time  
    effective_sample_count  
  
  blocks.py  
    FrameInterval  
    CompleteFrameBlockPolicy  
    CompleteFrameBlockPlan  
    contiguous_frame_runs  
    split_frame_interval  
    build_complete_frame_block_plan  
  
  assignment.py  
    BalancedAssignmentPlan  
    PurgedKFoldPolicy  
    PurgedFold  
    PurgedKFoldPlan  
    assign_balanced_round_robin  
    purge_neighbor_positions  
    build_purged_kfold_plan
```

All records are immutable, canonically serialized, and protected by SHA-256 content digests. The digests are deterministic content identities, not public-key signatures.

Theoretical background

Correlated trajectory data

For a stationary scalar sequence x_t , the normalized autocorrelation is

$$\rho(k) = \frac{\text{Cov}(x_t, x_{t+k})}{\text{Var}(x_t)}.$$

mdstats uses the convention

$$\tau_{\text{int}} = \frac{1}{2} + \sum_{k=1}^{k^*} \rho(k).$$

An uncorrelated sequence therefore has $\tau_{\text{int}} = 1/2$. The corresponding effective sample count is

$$N_{\text{eff}} = \min\left(N, \frac{N}{2\tau_{\text{int}}}\right).$$

Block averaging is a standard way to prevent correlated samples from being mistaken for independent evidence [1]. Geyer’s initial-positive-sequence rule provides a deterministic truncation rule for noisy autocorrelation estimates [2].

Exact estimator

AutocorrelationPolicy version mdstats.sampling-autocorrelation.initial-positive-sequence.2026-07.v1 requires:

input	finite one-dimensional float64 sequence
autocovariance	FFT-computed
lag normalization	divide lag-k sum by N-k (unbiased autocovariance)
truncation	initial positive sequence of adjacent rho pairs
tau floor	0.5 stored frames
tau ceiling	N/2 stored frames
short sequence	tau = 0.5, status = insufficient
constant sequence	tau = 0.5, status = constant

For lags (1, 2), (3, 4), and so on, the estimator accumulates a pair only while

$$\rho(2m - 1) + \rho(2m) > 0.$$

When the final available lag is unpaired, it is included only when its correlation is positive. This is the exact historical STAT1/SAMP0 numerical oracle.

No autocorrelation is computed across a gap, continuation reset, or excluded interval. The caller first divides the frame axis into contiguous runs. Observable values outside the eligible frame set are ignored and may be missing or nonfinite; every eligible value must be finite.

Autocorrelation data contract

AutocorrelationPolicy

```

policy_version
estimator
minimum_observations
minimum_tau_frames
maximum_tau_fraction
variance_floor
signature

```

Only `fft_unbiased_initial_positive_sequence` is accepted in DATA1.

AutocorrelationEstimate

policy_signature
status
observation_count
mean
variance
autocorrelation_time_frames
effective_sample_count
truncation_lag
included_positive_sequence_terms
notes
signature

status is one of:

estimated
constant
insufficient

The estimate is expressed in stored-frame units. Conversion to physical time is a higher-level operation using the frame-time contract.

Contiguous runs and complete-frame blocks

Frame ownership

A frame is indivisible. All atoms and labels from one configuration remain in one interval and later in one statistical role. DATA1 operates only on integer frame indices, so atom multiplicity cannot inflate the number of independent time samples.

FrameInterval

A frame interval is half-open:

$$[s, e) = \{s, s + 1, \dots, e - 1\}.$$

It requires $0 \leq s < e$.

Contiguous-run rule

Given strictly increasing unique eligible indices, `contiguous_frame_runs` creates maximal intervals separated whenever

$$i_{n+1} - i_n \neq 1.$$

No block may cross such a gap.

Resolved block length

For every observable j and every contiguous run r , estimate $\tau_{j,r}$. Define

$$\tau_{\max} = \max_{j,r} \tau_{j,r}.$$

The automatic target is

$$L_{\text{corr}} = \max(1, \lceil m\tau_{\max} \rceil),$$

and the resolved target is

$$L = \max(L_{\min}, L_{\text{corr}}).$$

An explicit length may override L . The plan records when that override is shorter than the correlation-derived target; downstream adequacy remains fail-closed.

Balanced all-frame split

For a run of length $n > L$, the number of blocks is

$$B = \max\left(1, \left\lfloor \frac{n}{L} \right\rfloor\right).$$

All frames are retained. Writing $n = qB + r$, the first r blocks contain $q + 1$ frames and the remainder contain q . No tail is discarded.

CompleteFrameBlockPlan

policy_signature
 eligible_frame_indices
 contiguous_runs
 block_intervals
 observable_autocorrelation_times_frames
 maximum_autocorrelation_time_frames
 decorrelation_target_length_frames
 resolved_block_length_frames
 explicit_length_override
 notes
 signature

The plan validates that runs and blocks each cover every eligible frame exactly once and contain no ineligible frame.

Deterministic assignment

Balanced round robin

For ordered items $b_0, \dots, b_{N-1}$ and ordered labels $\ell_0, \dots, \ell_{K-1}$, assignment is

$$b_i \mapsto \ell_{i \bmod K}.$$

The load difference between any two labels is at most one. Input order is scientific provenance; no hidden random shuffle occurs.

BalancedAssignmentPlan stores:

item_ids
 labels
 ordered_assignments
 strategy
 signature

Purged folds

For $\text{resolved_fold_count} = \min(\text{requested_fold_count}, N)$, fold k evaluates items whose positions satisfy

$$i \bmod K = k.$$

A purge radius h removes every non-evaluation item within h ordered positions of an evaluation item. The remaining items form the training role. A fold with no remaining training item is recorded as omitted rather than silently accepted.

PurgedKFoldPlan stores:

```
policy_signature
ordered_item_ids
resolved_fold_count
feasible PurgedFold records
omitted_fold_indices
signature
```

Each PurgedFold classifies every item exactly once as training, evaluation, or purged.

Stage 11 compatibility refactor

DATA1 replaces duplicate private numerical helpers in:

```
mdstats.io.production_regimes
mdstats.io.sampling_crossfit
```

The following existing contracts remain unchanged:

- public Stage 11 classes and function names;
- policy schemas and signatures;
- JSON payloads;
- block boundaries;
- domain assignments;
- nested fold roles;
- autocorrelation times;
- effective sample counts;
- acceptance outcomes.

The compatibility gate compares complete serialized STAT1 and SAMP0 artifacts before and after the refactor. Stage 11 serialized values must be byte-for-byte identical for the frozen fixtures.

Failure behavior

The primitives reject:

- empty autocorrelation sequences;
- multidimensional observables or nonfinite values on eligible frames;
- negative, duplicate, or unsorted frame indices;
- invalid or empty intervals;
- unsupported estimator, remainder, or assignment strategies;
- duplicate item IDs or labels;
- purge positions outside the item axis;
- tampered serialized payloads.

They do not infer missing values, reorder frames, bridge gaps, discard remainders, or create a random seed.

Focused acceptance tests

MLFF-DATA1 requires:

1. exact autocorrelation parity with the frozen STAT1/SAMP0 implementation over short, constant, random, and strongly correlated sequences;
2. complete-frame coverage across multiple temporal gaps;
3. exact historical balanced-remainder boundaries;

4. disjoint and complete balanced assignments;
5. disjoint training/evaluation/purge roles in every fold;
6. serialization round trip and tamper rejection;
7. exact pre/post-refactor STAT1 and SAMP0 JSON equality;
8. preservation of all existing Stage 11 focused tests;
9. public export and clean-wheel import checks.

References

[1] H. Flyvbjerg and H. G. Petersen, “Error Estimates on Averages of Correlated Data,” *Journal of Chemical Physics* **91**, 461-466 (1989). <https://doi.org/10.1063/1.457480>.

[2] C. J. Geyer, “Practical Markov Chain Monte Carlo,” *Statistical Science* **7**, 473-483 (1992). <https://doi.org/10.1214/ss/1177011137>.